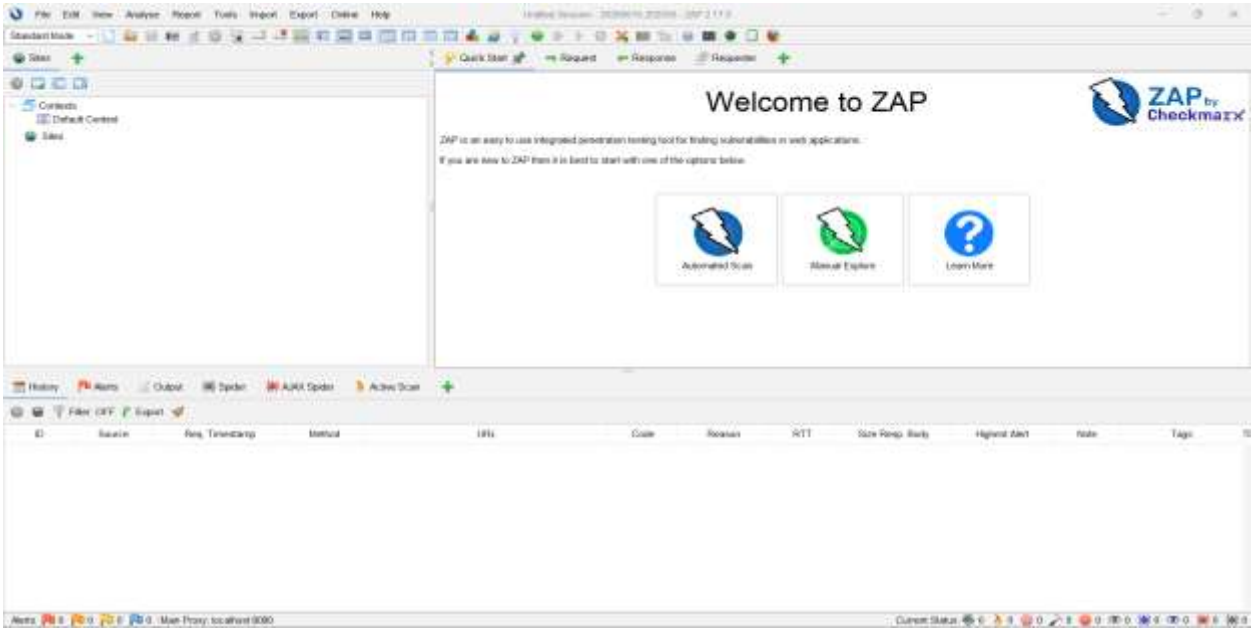
Security Validation of a Web Application using OWASP ZAP

Use demo website

OWASP ZAP Home Screen



Run Automated Security Scan

Objective

The purpose of this task was to perform an automated security scan on a vulnerable web application using OWASP ZAP.

Target Website

Target URL: https://demo.owasp-juice.shop/#/

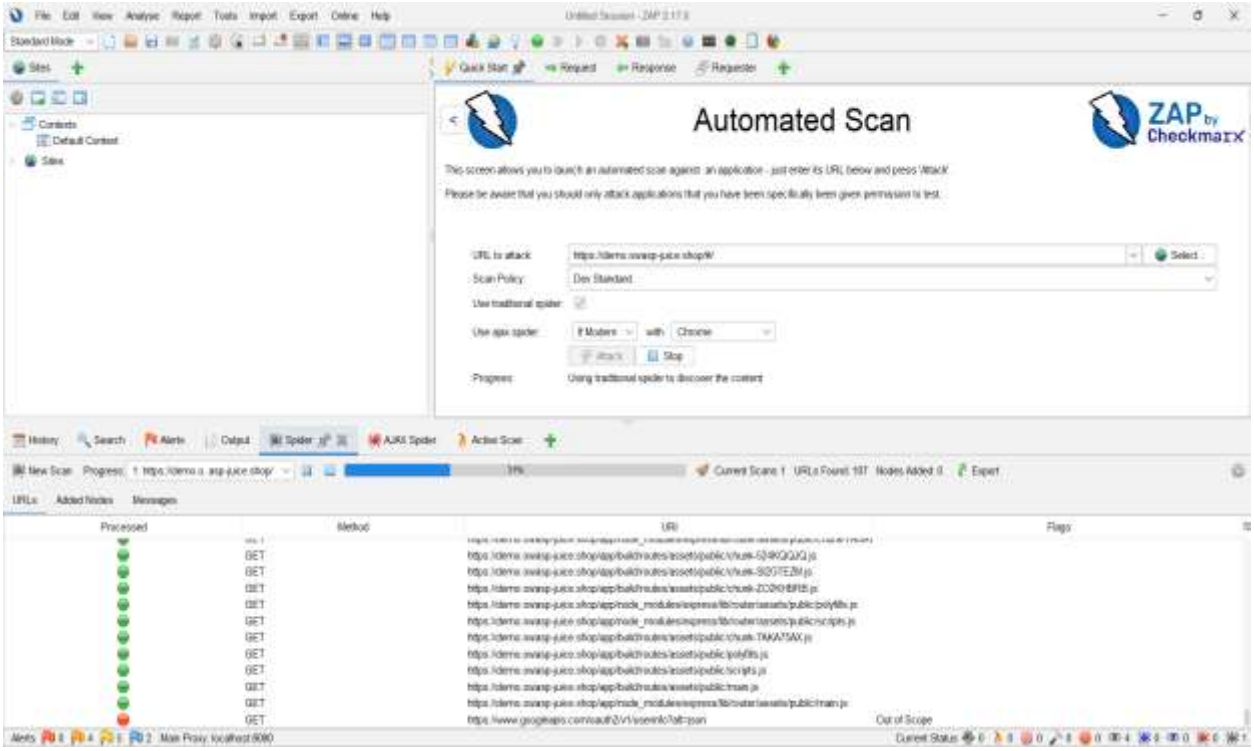
Steps Performed

1. Opened ZAP 2.17.0.
2. Selected: Quick Start → Automated Scan
3. Entered the target URL.
4. Clicked on the “Attack” button.
5. Waited for the scan to complete.
6. Reviewed the detected vulnerabilities.

Result

The automated scan completed successfully and multiple security vulnerabilities were detected.

Scaning Proccess:



Scan Results

Identify Vulnerabilities

Vulnerability Details

1. Cross-Domain Misconfiguration (Systemic)

- **Risk Level:** Medium
- **Affected Page:** https://demo.owasp-juice.shop/chunkGNBEOV4E.js
- **Description:** Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

2. Session ID in URL Rewrite (Systemic)

- **Risk Level:** Medium
- **Affected Page:** https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=websocket&sid=PnN8s_8kH-Em05pcAATC
- **Description:** URL rewrite is used to track user session ID. The session ID may be disclosed via cross-site referer header. In addition, the session ID might be stored in browser history or server logs.

3. Private IP Disclosure

- **Risk Level:** Low
- **Affected Page:** https://demo.owasp-juice.shop/rest/admin/application-configuration
- **Description:** A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.

4. Timestamp Disclosure – Unix (Systemic)

- **Risk Level:** Low
- **Affected Page:** https://demo.owasp-juice.shop/chunk-SI2GTEZM.js
- **Description:** A timestamp was disclosed by the application/web server. - Unix

Vulnerability Report Table

Vulnerability	Severity	Affected Page	Description
Cross-Domain Misconfiguration	Medium	https://demo.owasp-juice.shop/chunkGNBEOV4E.js	CORS misconfiguration may allow unauthorized cross-origin data access in the browser.
Session ID in URL Rewrite	Medium	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=websocket&sid=PnN8s_8kH-Em05pcAATC	Session ID in URL may leak via referrer, browser history, or server logs.
Private IP Disclosure	Low	https://demo.owasp-juice.shop/rest/admin/application-configuration	Private IP/hostname exposed in response may help attackers target internal systems.
Timestamp Disclosure - Unix	Low	https://demo.owasp-juice.shop/chunk-SI2GTEZM.js	A timestamp was disclosed by the application/web server. - Unix

Recommend Security Improvements

Vulnerability	Recommendation
Cross-Domain Misconfiguration	Ensure sensitive data is not accessible without authentication. Configure the Access-Control-Allow-Origin header to allow only specific trusted domains, or remove CORS headers entirely to enforce the browser's Same Origin Policy (SOP).
Session ID in URL Rewrite	Move session IDs from URLs into secure cookies. For stronger security, use a combination of both cookie-based and URL rewrite session management to reduce exposure risk.
Private IP Disclosure	Remove all private IP addresses from HTTP response bodies. Replace HTML or JavaScript comments containing sensitive info with server-side comments (JSP/ASP/PHP) that are not visible to client browsers.
Timestamp Disclosure - Unix	Manually verify that exposed timestamp data is not sensitive. Ensure timestamps cannot be collected or aggregated in a way that reveals exploitable patterns such as predictable tokens or session values.

Additional Security Recommendations

#	Recommendation	Description
1	**Implement Content Security Policy (CSP)**	Add a Content-Security-Policy header to all responses to control which sources of content (scripts, styles, images) are allowed to load, preventing XSS attacks.
2	**Enable HTTP Strict Transport Security (HSTS)**	Add the Strict-Transport-Security header to force browsers to only communicate over HTTPS, preventing downgrade attacks and cookie hijacking.
3	**Set X-Content-Type-Options Header**	Add X-Content-Type-Options: nosniff header to prevent browsers from MIME-type sniffing, reducing the risk of malicious file execution.
4	**Add X-Frame-Options Header**	Configure X-Frame-Options: DENY or SAMEORIGIN to prevent the site from being embedded in iframes, protecting users from Clickjacking attacks.
5	**Hide Server Version Information**	Configure the server to remove or mask the Server HTTP response header to avoid exposing software name and version to potential attackers.
6	**Implement Strong Password Policy**	Enforce minimum password length, complexity, and account lockout after failed attempts to protect against brute force and credential stuffing attacks.
7	**Enable Multi-Factor Authentication (MFA)**	Require a second form of verification during login to significantly reduce the risk of unauthorized access even if credentials are compromised.
8	**Regular Security Scanning & Patching**	Schedule automated vulnerability scans regularly and keep all server software, frameworks, and libraries up to date to address newly discovered vulnerabilities.

Validation Reflection

Q1: Why is security validation important?

Security validation is important because it helps find weaknesses and vulnerabilities in a software system before attackers can take advantage of them. It helps protect sensitive data, improves overall system security, and reduces possible risks. It also increases user trust in the system.

Q2: What is the difference between functional correctness and security correctness?

Functional correctness means checking whether the software is working according to the given requirements and is performing all expected tasks properly. Security correctness means checking whether the software is safe from attacks, unauthorized access, and any kind of misuse or security threats.

Q3: How does OWASP ZAP help validation engineers?

OWASP ZAP helps validation engineers by automatically checking web applications for security weaknesses. It finds issues like XSS, missing security headers, and injection problems. It also creates reports that help engineers understand risks and improve the security of applications.

In our scan, OWASP ZAP found several vulnerabilities in the OWASP Juice Shop application, such as Cross-Domain Misconfiguration, Session ID in URL Rewrite, Private IP Disclosure, and some other security issues.
.

Conclusion

In this lab task, security validation of the OWASP Juice Shop web application was carried out using OWASP ZAP 2.17.0. During the scan, a total of 11 alerts were detected. These included vulnerabilities such as Cross-Domain Misconfiguration, Session ID in URL Rewrite, Private IP Disclosure, and Timestamp Disclosure.

Each issue was studied according to its risk level, and recommendations were provided based on OWASP ZAP suggestions to improve the security of the application. This lab showed how automated security scanning helps in finding real-world vulnerabilities. It also highlighted the importance of security validation in the overall software verification and validation process.